GOSIM

GOSIM Paris 2026

GOSIM Paris 2026

May 5-6, 2026 Station F, Paris

GOSIM

GOSIM Paris 2026

One Year of llama.cpp

Milestones and Future Plans

Self-introduction

Xuan-Son Nguyen
SWE @ Hugging Face
Core maintainer of llama.cpp

Website: ngxson.com
Github / X: @ngxson

My website

What is llama.cpp?

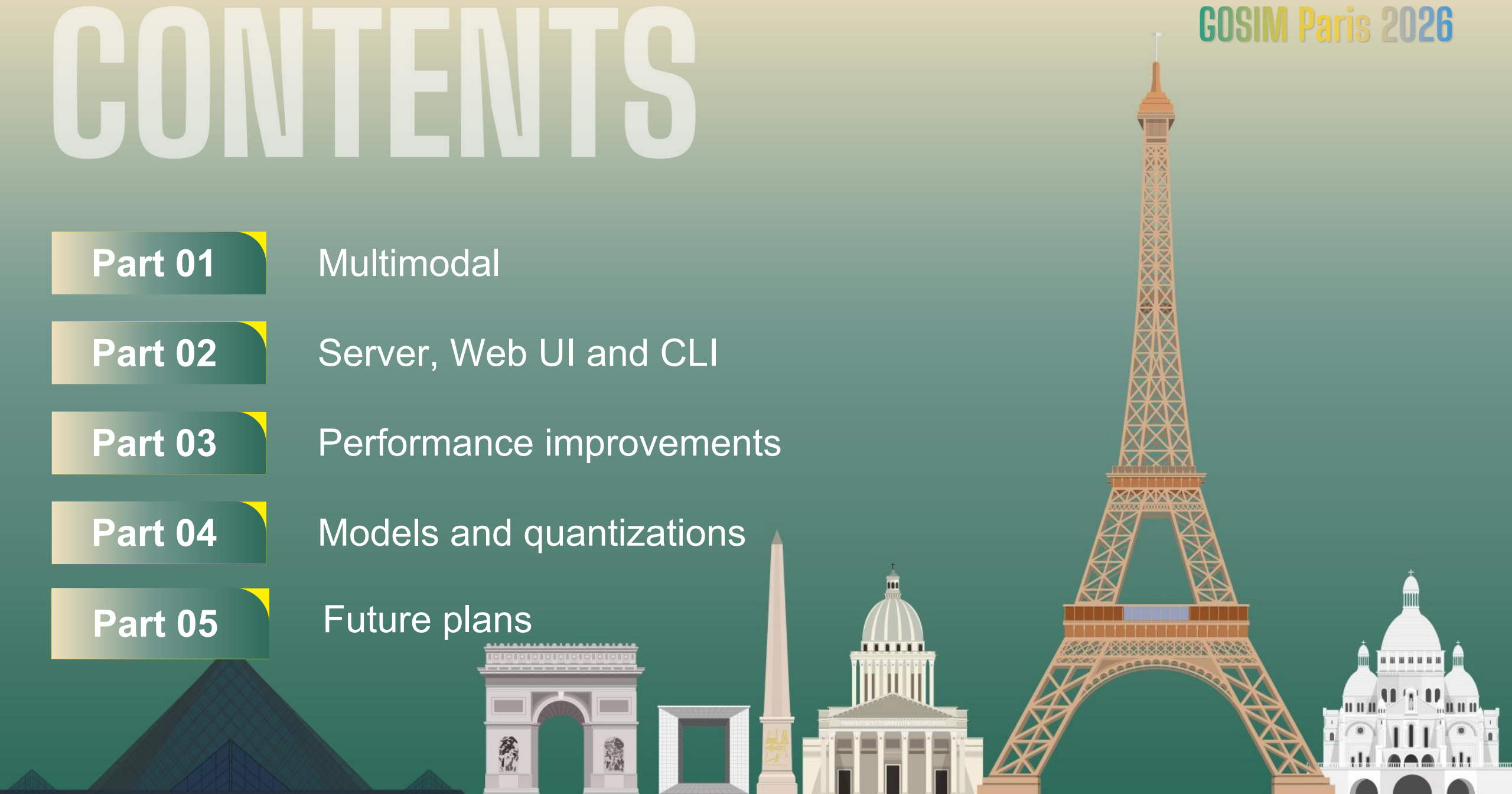
→ An inference engine written predominantly in C++
→ Cross-platform
→ Support a wide variety of hardware: CPU, GPU (Nvidia, AMD, Intel, Snapdragon), Metal

How to use it?

→ Follow the quick started guide: https://github.com/ggml-org/llama.cpp/#quick-start

CONTENTS

GOSIM Paris 2026



01

Multimodal

Pre-context: Llama.cpp has initial support for vision input (llava model) since the very beginning.
However, it's not simple to use

LLaVA 1.5

1. Clone a LLaVA and a CLIP model (available options). For example:

```
git clone https://huggingface.co/liuhaotian/llava-v1.5-7b

git clone https://huggingface.co/openai/clip-vit-large-patch14-336
```

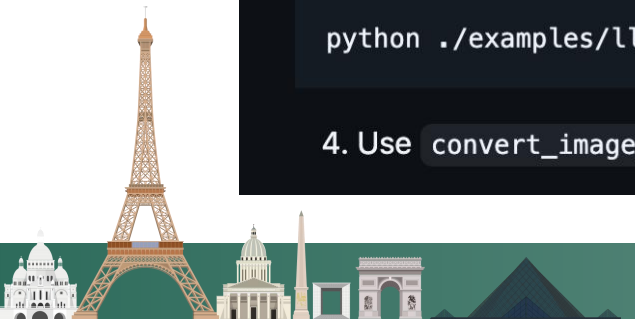
2. Install the required Python packages:

```
pip install -r examples/llava/requirements.txt
```

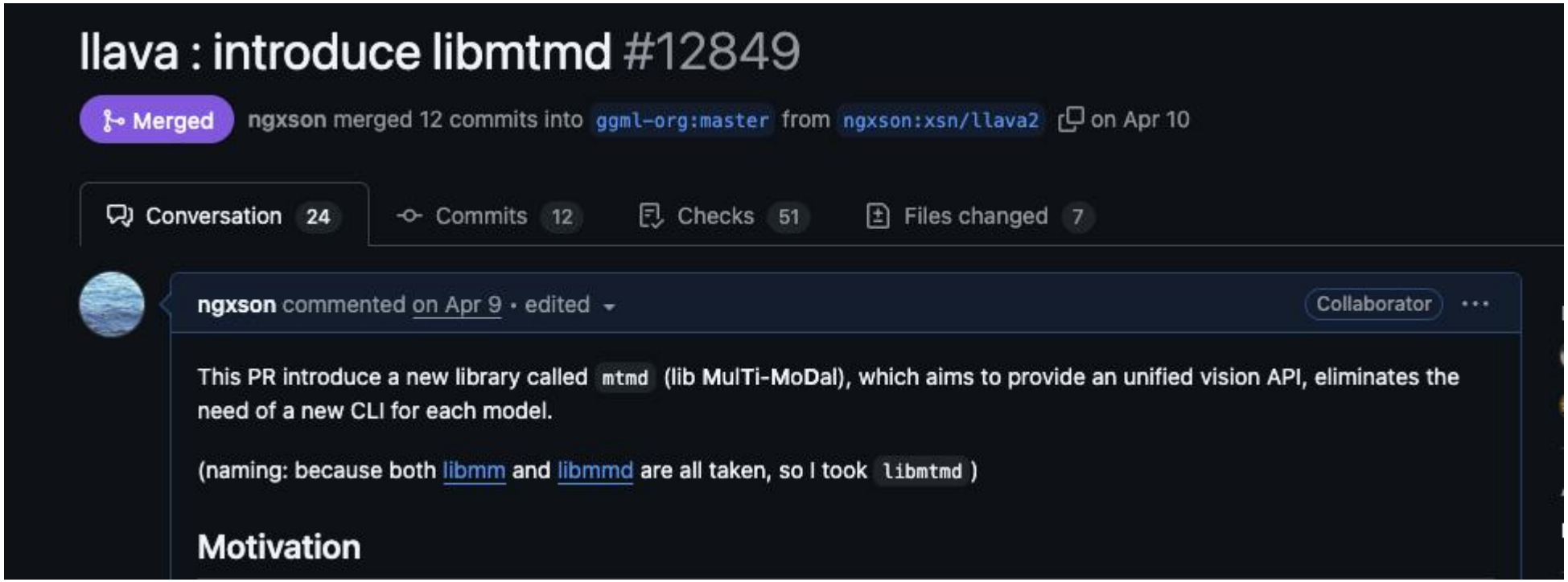
3. Use `llava_surgery.py` to split the LLaVA model to LLaMA and multimodel projector constituents:

```
python ./examples/llava/llava_surgery.py -m ../llava-v1.5-7b
```

4. Use `convert_image_encoder_to_gguf.py` to convert the LLaVA image encoder to GGUF:



→ Mid 2025, I introduced a sub-module inside llama.cpp called mtmd (multimodal)



Mtmd aims for:
- Stable API for multimodal processing
- Support multiple modalities
- Easy for developers and for users

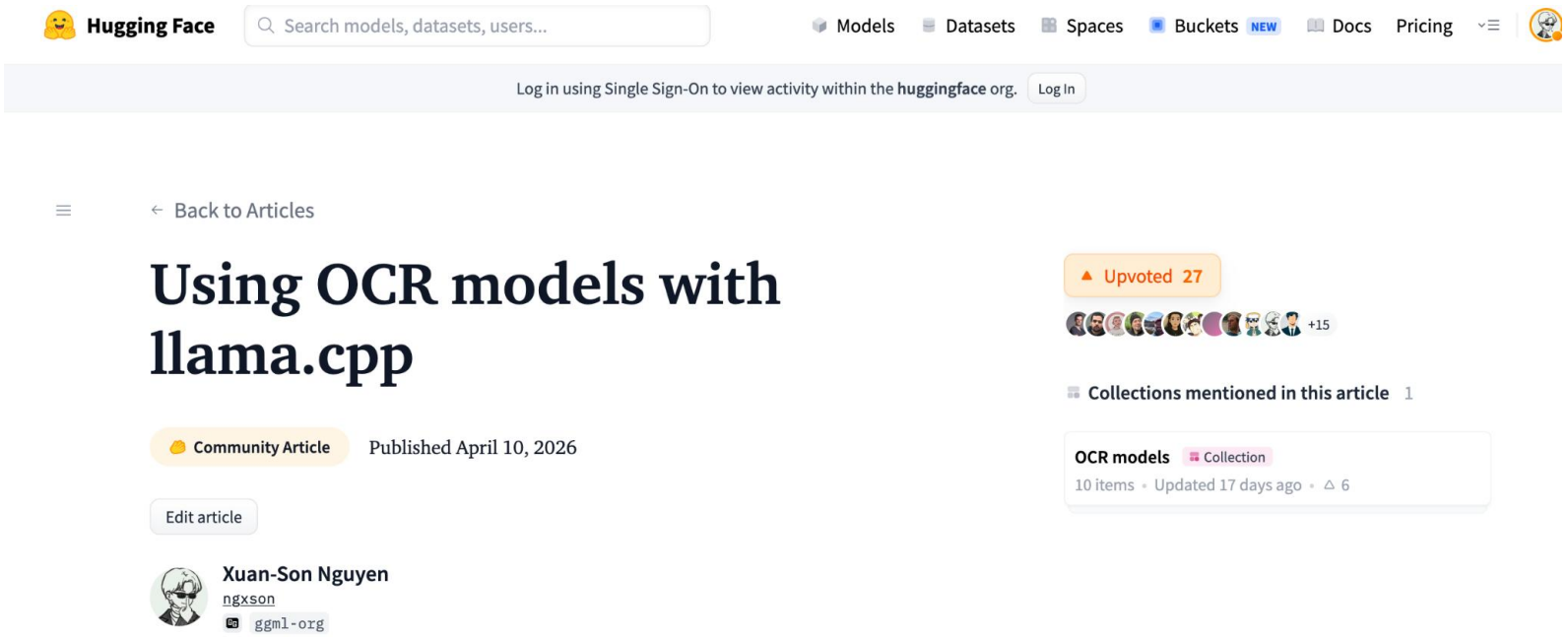
More technical details?
Watch my talk at GOSIM 2025:
https://www.youtube.com/watch?v=D3qlSFta4Hc

Youtube video

(Demo in next section)

Support modalities: audio input, vision input
Work-in-progress: audio output, video input
OCR / ASR capabilities

Ref OCR blog post: https://huggingface.co/blog/ggml-org/using-ocr-models-with-llama-cpp



GOSIM

GOSIM Paris 2026

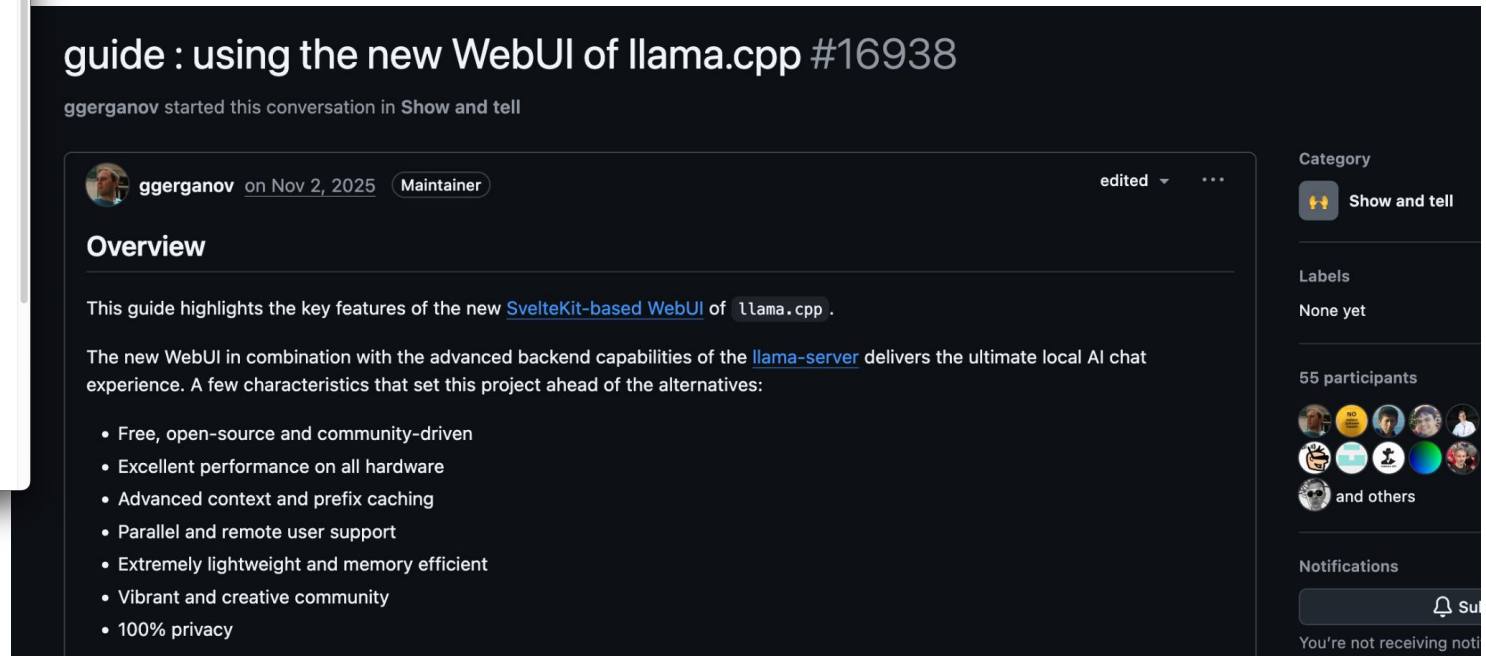
02

Server, Web UI and CLI

SERVER

- Multimodal support via openai-compatible api
- Streamed reasoning and tool calling (+ parallel tool calls) possible
  (thanks to @pwilkin and @aldehir for chat parsing infrastructured)
- Anthropic-compatible api: https://huggingface.co/blog/ggml-org/anthropic-messages-api-in-llamacpp
- Server “router” mode allow using multiple models via the same llama-server:
  https://huggingface.co/blog/ggml-org/model-management-in-llamacpp

New Web UI overhaul thanks to Alek (@allozaur)
Link to guide: https://github.com/ggml-org/llama.cpp/discussions/16938



New CLI, simple to use; Example command:

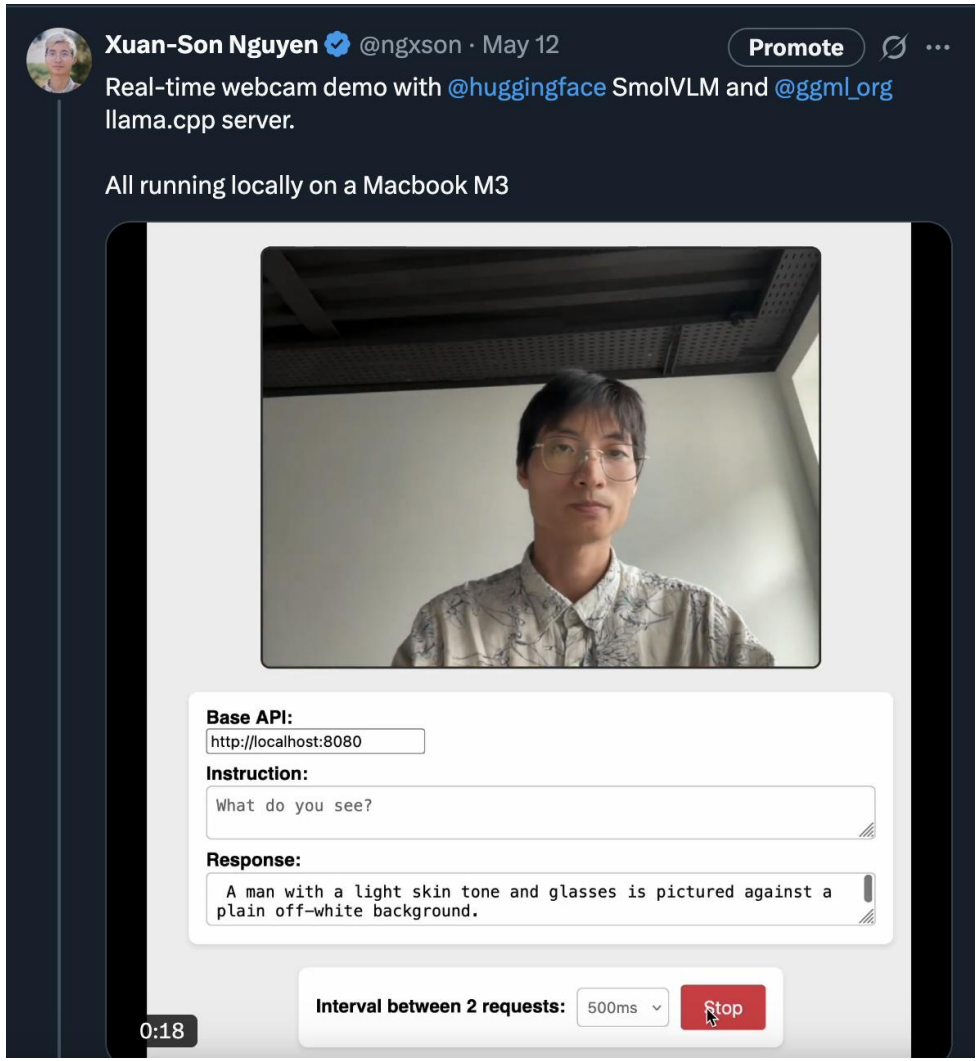
```
llama-cli -hf ggml-org/gemma-4-26B-A4B-it-GGUF
```

Server, Web UI and CLI

Demo Web UI / server:

- Start a model
- Upload image
- Watch the API calls
- MCP servers

See more: Real-time webcam recognition
https://x.com/ngxson/status/1921980096421806127



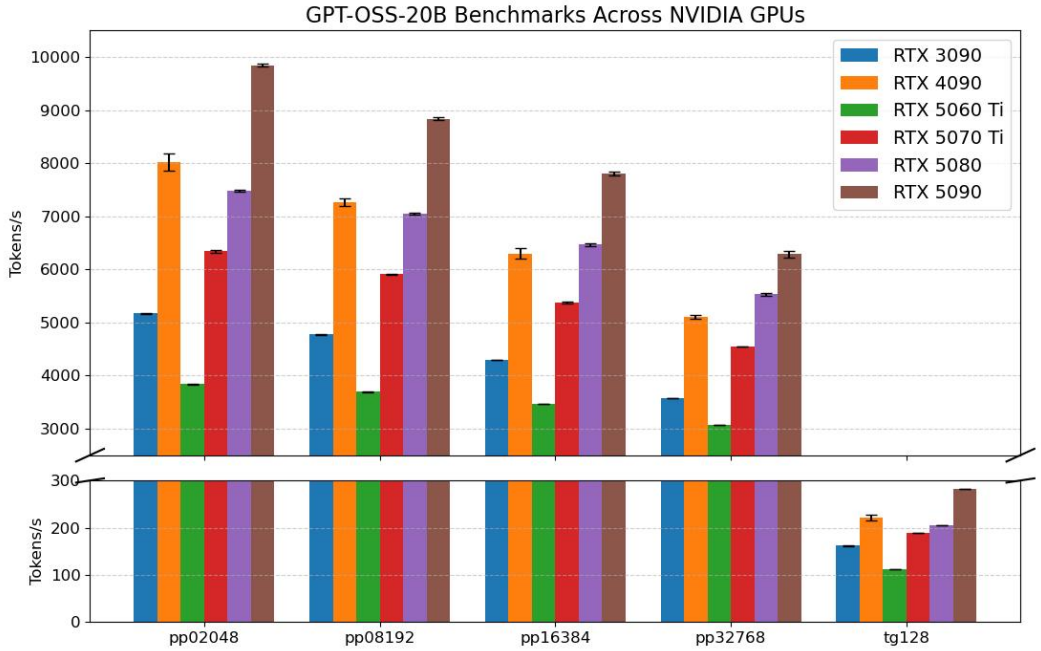
03

Performance improvements

Many backend improvements, I cannot list all - most recently:

- Various improvements in Flash Attention kernel across all backends
- Cross-backend tensor parallelism (thanks to @JohannesGaessler)
- Improved support for multi-GPU setup
- New backends: WebGPU, VirtGPU, ZenDNN
- MXFP4 and NVFP4 support

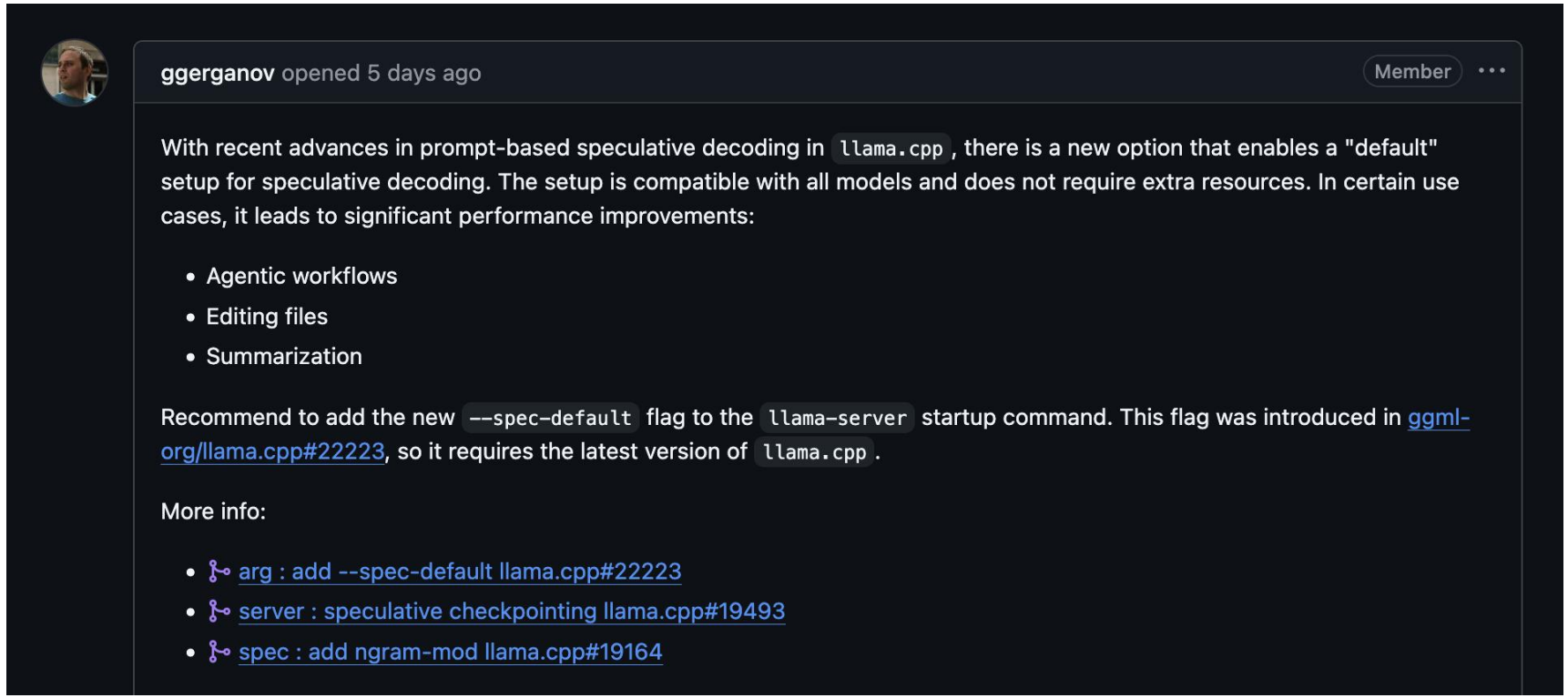
Example: GPT-OSS benchmarks:
https://github.com/ggml-org/llama.cpp/discussions/15396



Inference improvements:

- Self-speculative decoding (via n-gram)
  https://github.com/ggml-org/LlamaBarn/issues/83

- Better context re-using for recurrence models via context checkpoints (Qwen 3.5 / Qwen 3.6)



ggerganov opened 5 days ago — Member

With recent advances in prompt-based speculative decoding in `llama.cpp`, there is a new option that enables a "default" setup for speculative decoding. The setup is compatible with all models and does not require extra resources. In certain use cases, it leads to significant performance improvements:

- Agentic workflows
- Editing files
- Summarization

Recommend to add the new `--spec-default` flag to the `llama-server` startup command. This flag was introduced in ggml-org/llama.cpp#22223, so it requires the latest version of `llama.cpp`.

More info:

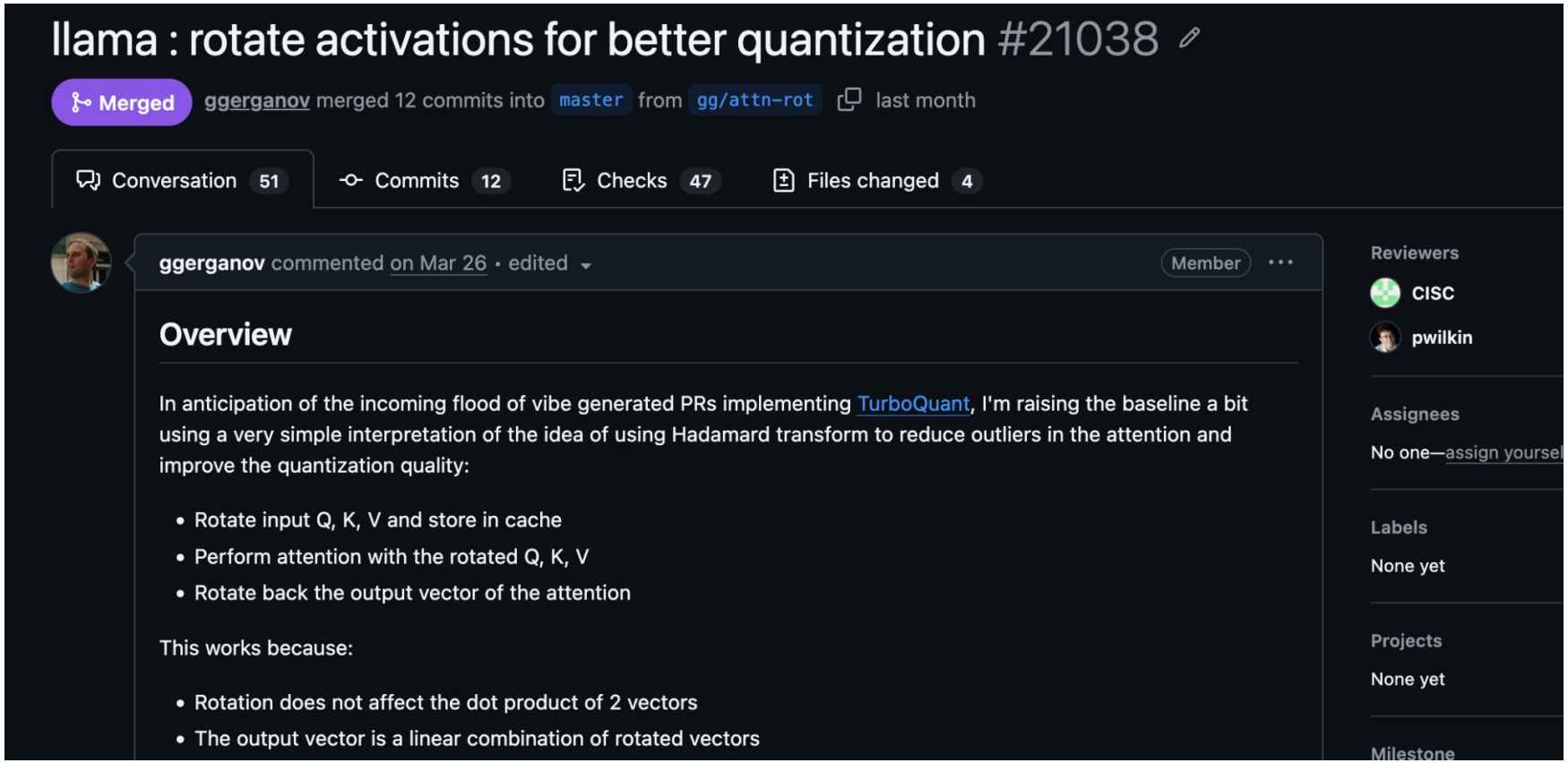
- arg : add --spec-default llama.cpp#22223
- server : speculative checkpointing llama.cpp#19493
- spec : add ngram-mod llama.cpp#19164

04

Models and quantizations

- Model SOTA models are supported, including: Qwen 3.x, Gemma 4, Kimi-K2.x, Mistral Ministral / Mistral Small, GPT-OSS, GLM family
- To search for compatible, go to Hugging Face and search for “(model name) GGUF”

- New quantization: MXFP4 and NVFP4
- KV cache quantizations (Q4, Q8) with Hadamard transformation



05

Future plans

- Add Agent mode to CLI and Web UI
- Multimodal: video input, audio output
  (Longer in the future, maybe add image output)
- Multi-token prediction support
- Backend and speed optimizations; quantization improvements…

GOSIM

GOSIM Paris 2026

Thanks!

My website